

This program imports `get_formatted_name()` from `name_function.py`. The user can enter a series of first and last names and see the formatted full names that are generated:

---

Enter 'q' at any time to quit.

Please give me a first name: **janis**

Please give me a last name: **joplin**

Neatly formatted name: Janis Joplin.

Please give me a first name: **bob**

Please give me a last name: **dylan**

Neatly formatted name: Bob Dylan.

Please give me a first name: **q**

---

We can see that the names generated here are correct. But say we want to modify `get_formatted_name()` so it can also handle middle names. As we do so, we want to make sure we don't break the way the function handles names that have only a first and last name. We could test our code by running `names.py` and entering a name like Janis Joplin every time we modify `get_formatted_name()`, but that would become tedious. Fortunately, `pytest` provides an efficient way to automate the testing of a function's output. If we automate the testing of `get_formatted_name()`, we can always be confident that the function will work when given the kinds of names we've written tests for.

## ***Unit Tests and Test Cases***

There is a wide variety of approaches to testing software. One of the simplest kinds of test is a unit test. A *unit test* verifies that one specific aspect of a function's behavior is correct. A *test case* is a collection of unit tests that together prove that a function behaves as it's supposed to, within the full range of situations you expect it to handle.

A good test case considers all the possible kinds of input a function could receive and includes tests to represent each of these situations. A test case with *full coverage* includes a full range of unit tests covering all the possible ways you can use a function. Achieving full coverage on a large project can be daunting. It's often good enough to write tests for your code's critical behaviors and then aim for full coverage only if the project starts to see widespread use.

## ***A Passing Test***

With `pytest`, writing your first unit test is pretty straightforward. We'll write a single test function. The test function will call the function we're testing, and we'll make an assertion about the value that's returned. If our assertion is correct, the test will pass; if the assertion is incorrect, the test will fail.